



A hybrid approach for Bangla sign language recognition using deep transfer learning model with random forest classifier

Sunanda Das^a, Md. Samir Imtiaz^a, Nieb Hasan Neom^a, Nazmul Siddique^{b,*}, Hui Wang^c

^a Department of Computer Science and Engineering, Khulna University of Engineering & Technology, Khulna 9203, Bangladesh

^b School of Computing, Engineering and Intelligent Systems, Ulster University, Londonderry BT48 7JL, UK

^c School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, Belfast, BT9 5BN, UK

ARTICLE INFO

Keywords:

Bangla sign language
Deep learning
Convolutional neural network (CNN)
Transfer learning
Character recognition
Digit recognition

ABSTRACT

Sign language is the comprehensive medium of mass communication for hearing and speaking impaired individuals. As they cannot speak or hear, they are not able to use sound or vocal signals as an information medium for their communication. Rather, they are bound to exchange visual signals to express their feeling in their day-to-day life. For this, they use various body language mainly hand gestures as sign language. Sign language fundamentals can be largely divided into two parts namely digits (numerals) and characters (alphabetical). In this paper, we proposed a hybrid model consisting of a deep transfer learning-based convolutional neural network with a random forest classifier for the automatic recognition of Bangla Sign Language (numerals and alphabets). The overall performance of the presented system is verified on 'Ishara-Bochon' and 'Ishara-Lipi' datasets. 'Ishara-Bochon' and 'Ishara-Lipi' are datasets of isolated numerals and alphabets respectively which are the first complete multipurpose open-access dataset for Bangla Sign Language (BSL). Besides, we also proposed a background elimination algorithm that removes unnecessary features from the sign images. Along with the proposed background elimination technique, the system is able to achieve accuracy, precision, recall, f1-score values of 91.67%, 93.64%, 91.67%, 91.47% for character recognition and 97.33%, 97.89%, 97.33%, 97.37% for digit recognition respectively. The detailed experimental analysis assures the feasibility and effectiveness of the proposed system for BSL recognition.

1. Introduction

Sign language has been developed for speaking and hearing-impaired people to communicate effectively. A language is a system of communication that comprises a collection of sounds and written symbols used by a population. But, hearing and speaking impaired people cannot use that language to communicate, rather they use sign language. There is no standardized universal form of sign languages and thus sign languages from different countries or regions are not mutually intelligible with each other.

According to World Health Organization (WHO), about 466 million people (432 million adults and 34 million children) globally are suffering from some kind of deafness and hearing disability that makes up to 6.1% of the world's population and by the year 2050, the number is estimated to go to over 900 million which is about one in every ten people (W.H.O., 2021). These numbers show the importance of sign language and machine-based sign language translation systems. A lot of research work has been done on the recognition of sign languages.

In the recent years, Bengali Sign Language (BSL) recognition has also received a lot of attention. Machine learning and lately deep learning have achieved great improvement in the field of recognition and classification (Anand, Urolagin, & Mishra, 2021; Machiraju, Urolagin, Mishra, & Sharma, 2021; Singh, Mishra, Urolagin, & Sharma, 2021). With the invention and development of powerful Convolutional Neural Network (CNN) models, it has reached new heights of success. Applying CNN models in the automatic recognition of sign language is a popular choice among researchers nowadays. A sign language recognition and translation system can break down the communication barrier between normal people and hearing-impaired people by creating a communication bridge. Allowing hearing-impaired people to participate in social, economic, and political activities can help them become valuable assets to the society.

Despite the fact that Bangla is the world's seventh most spoken language, with 265 million speakers (Berlitz, 2021), there are limited studies and prospects for working in the automatic detection of Bengali

* Corresponding author.

E-mail addresses: sunanda@cse.kuet.ac.bd (S. Das), samirimtiaz1234@gmail.com (M.S. Imtiaz), niebhasanneom0@gmail.com (N.H. Neom), nh.siddique@ulster.ac.uk (N. Siddique), h.wang@qub.ac.uk (H. Wang).

<https://doi.org/10.1016/j.eswa.2022.118914>

Received 2 June 2022; Received in revised form 15 August 2022; Accepted 24 September 2022

Available online 30 September 2022

0957-4174/© 2022 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

sign language. To make a contribution to this field, we have developed a deep learning-based BSL recognition system that can accurately categorize BSL characters and digits from sign images. Many of the previous research (Haque, Das, & Kaspy, 2019; Rahman, Abdullah, Mondal, et al., 2012; Uddin, Arko, Tabassum, Trisha, & Ahmed, 2017) used a limited amount of data due to the unavailability of large-scale open-source datasets in sign language recognition and performed very inadequately. We employed data augmentation and several pre-trained models to overcome the lack of limited samples in datasets, as pre-trained models can effectively deal with minimal datasets and increase model performance (Das, Fime, Siddique, & Hashem, 2021; Kieffer, Babaie, Kalra, & Tizhoosh, 2017). The significant contributions of the research work are:

- Employment of the proposed background elimination algorithm with the help of different morphological operations and adaptive Gaussian Threshold technique which dynamically decides threshold value for a pixel by considering certain small neighborhoods around it so that an optimum threshold value can be determined for different regions depending on local background pixels.
- Employment of transfer learning with the help of weight transferring from different backbone networks (VGG16, VGG19, InceptionV3, Xception, ResNet50) pre-trained on ImageNet to handle small-sized datasets and hybridization of transfer learning and Random Forest (RF) classifier. Because the RF only considers a small subset of features at each split of the tree, it can deal with data having higher dimensions. If a single decision tree contains high variance, the RF bagging or bootstrap aggregating method combines a lot of weak learners into strong learners, which averages out the variance.
- Utilization of grid search algorithm on different batch sizes for each model separately to come up with a fine-tuned batch size which is necessary to generate optimum results from the proposed hybrid architecture. The proposed architecture is trained with the mini-batch technique which offers a regularizing effect, updates the network's weights after each propagation, and reduces the generalization error while providing stable convergence and satisfactory test performance which makes the batch size an important hyper-parameter.
- Improvement of evaluation parameters such as accuracy, precision, recall, and f1-score compared to other existing recognition methods.

The rest of the paper is arranged as follows: A very extensive overview and analysis of previous research in this domain are provided in Section 2. Section 3 describes the overall system methodology along with the proposed background elimination module. In Section 4, the evaluation metrics are presented. After that, Section 5 illustrates the detailed experimental analysis of this study. Finally, in Section 6, the study's conclusions are drawn, along with the plan for further research.

2. Literature review

Sign language, like any natural language, has a wide range of expressions in different parts of the world. Over the years, some sign languages, such as American Sign Language (ASL), and Indian Sign Language (ISL) have gained prominence over others. To have a better understanding of the state of the art, we looked at several of these sign language recognition systems as well as certain Bengali Sign Language (BSL) recognition systems.

2.1. American Sign Language (ASL)

American Sign Language (ASL) is the primary sign language of hearing-impaired people in the United States and parts of Anglophone Canada. Fok, Cheng, and Ganganath (2015) proposed a novel real-time

ASL digit recognition system that senses data of hand gesture using two Leap Motion sensors (depth-sensors). Data of the two sensors are integrated using Kalman filters. The Hidden Markov Model is used for the recognition of hand gestures. The system performed better with higher evaluation metric values than other single depth-sensor based systems.

Zaki and Shaheen (2011) proposed a sign language recognition system that uses a novel combination of vision based features. Kurtosis position, principle component analysis and motion chain code features are combined to be mapped to four components of sign languages such as hand shape, place of articulation, hand orientation, and hand movement. Using HMM (Hidden Markov Model) as classifier, they achieved 89.1% recognition accuracy.

Kasukurthi, Rokad, Bidani, Dennisan, et al. (2019) introduced a neural network-based model for ASL alphabet identification from color images that can run on mobile devices. Prior to the conventional CNN architecture they used squeezeNet architecture (Iandola et al., 2016) that uses 1×1 filters for convolution operation which minimizes the number of trainable parameters and keeps the model size small to be deployable in mobile devices or cell phones. With their proposed model, they were able to achieve an accuracy of 83.29%.

Masood, Thuwal, and Srivastava (2018) used VGG16 model to classify 36 different hand gestures (26 alphabets and 10 numerals) of ASL from the dataset given by Barczak, Reyes, Abastillas, Piccio, and Susnjak (2011). They initialized the parameters of their model by transferring weights from the VGG16 network pretrained on ImageNet dataset which consists of more than a million images of 1000 categories. They modified the softmax layer of the VGG16 network according to their need as they had only 36 categories and trained the model using the ASL dataset. With this technique they obtained an impressive accuracy of 96%.

In Bilgin and Mutludoğan (2019), Bilgin et al. used Lenet and capsule networks separately for ASL character recognition. Then, they compared the outcome of capsule network with the outcome of LeNet-5 (LeCun, Bottou, Bengio, & Haffner, 1998) network architecture. They achieved 88% accuracy using capsule networks, where LeNet-5 was able to achieve 82% accuracy. Conventional CNN models like LeNet-5 evaluates features detected in the past layers as weighted sum in the following layers without preserving spatial information i.e. position and orientation related feature information about an image. Capsule networks successfully overcomes these limitations by treating features in the images as vectors in exchange for scalars which helps it preserve spatial information.

2.2. Indian Sign Language (ISL)

Despite popular belief, Indian Sign Language (ISL) is not a manual representation of spoken English or Hindi, and it has its own syntax. Sharma, Tomar, Mishra, and Chariar (2021) presented a transfer learning-based deep CNN architecture where they used the MobileNetV2 model pre-trained on ImageNet to recognize 35 classes of ISL symbols. They updated the weights of the unfrozen layers of the model by training them with the ISL dataset and achieved satisfactory performance on the test samples.

Sruthi and Lijiya (2019) proposed a vision-based recognition system for ISL static alphabets i.e. those alphabets that does not need continuous movement of hand or fingers. They used a deep learning-based approach to classify 24 sign categories and achieved a validation accuracy of 98.64%. Before training the model, they segmented the image dataset accordingly. For this, first they detected the face in the image with a face detection algorithm and removed it by replacing the corresponding pixels with black pixels. After that, they applied hand region segmentation using color segmentation algorithm and largest connected component algorithm. Then, the modified image dataset was fed into their 'signet model' for training. A real time system for numeral recognition of ISL was proposed by Sajanraj and Beena

(2018). They successfully developed a fast operating real-time system for ISL recognition and achieved a higher accuracy of 99.56%. The whole process is comprised of 4 basic steps. First is to take the images out of video frames. The next two steps include localization of hand in a rectangular boundary box and extraction of only the important features from the images using region of interest detection. Finally, these features were fed into a CNN model architecture to do necessary classification. The CNN model took motivation from LeNet architecture which is very popular model for digit recognition.

Rao, Syamala, Kishore, and Sastry (2018) presented a deep CNN architecture for ISL recognition. Parting away from most of the literature that classified ISL characters and numerals, they choose to classify 200 most frequently spoken ISL words. They captured the video using the selfie camera of mobile phone. They tried different CNN architectures and pooling methods and finally found out that a deep 14 layer CNN with stochastic pooling method works the best with an average recognition accuracy of 92.88%

2.3. Bangla Sign Language (BSL)

Bangla Sign Language (BSL) includes complex meaningful semantics that are generated by combining various components such as hand shapes, orientation, position, and movement. The developers of Ishara-Lipi dataset introduced a multilayer CNN model in Islam, Mousumi, Rabby and Hossain (2018) to classify BSL characters using their dataset containing 1800 sign character images. Bangla Sign characters are comparatively hard to classify as there are 36 different sign characters as opposed to 26 characters in ASL. Moreover, like ISL, these Bangla signs need the movement of both hands to represent a single character. With their very deep 16 layers model trained with ADAM optimizer and categorical crossentropy loss function, they got about 92% validation accuracy. The authors also created a separate BSL digit dataset named Ishara-Bochon (Islam, Mousumi, Rabby, Hossain and Abujar, 2018) containing 1075 sign digit images. They were able to achieve 95% accuracy by training the digits dataset into their proposed deep CNN model using stochastic gradient descent optimizer and categorical crossentropy loss function.

A complete system for BSL alphabet recognition was proposed by Rahman et al. in Rahman et al. (2012). It can take static hand gesture images as input and classify the gesture out of 36 BSL alphabet gestures with an average accuracy of 80.90% with high variance. As the classifier, they used standard feed-forward Artificial Neural Network (ANN) with sigmoid as activation function and mean square error as loss function. Uddin et al. (2017) introduced an image processing based model for BSL character recognition. At first, the RGB images were converted to YCbCr color components for skin detection. Then, features were extracted using bag of words method that involves feature extraction, visual vocabulary learning, feature quantization, and image representation using counts of visual words. These features were fed to support vector machine linear multiclass classifier. With their system, they achieved an accuracy of 86% while classifying 15 classes of BSL letters.

A dual purpose interpreter of one handed Bangla sign language that can do both text to sign language and sign language to text conversion was proposed by Rony, Saikat, Tanzeem, and Robi (2018). They used the idea of transfer learning and used InceptionV3 as the backbone CNN model. Jarman, Arshad, Alam, and Islam (2015) proposed a novel fingertip finder algorithm to identify one-handed static sign gestures of Bengali Sign Language including vowels, consonants and numerals from sign images of static background. After collecting necessary image data, in the preprocessing stage, they applied otsu's thresholding to come up with well segmented binary image. Median filtering and morphological operations were performed to remove noise in the segmented images. Then cropping was applied to keep only the region of interest. They used a multilayered feed forward neural network for classification purpose. They achieved maximum performance

with 27-nodes in the hidden layer with an average recognition rate of 88.69%.

Hossen et al. presented a CNN-based model for BSL recognition in Hossen, Govindaiah, Sultana, and Bhuiyan (2018). They worked with a diverse and natural color image dataset (Aziz, Wadud, Sultana, Hussain, & Bhuiyan, 2017) with varied background, color and lighting condition making it usable in real time systems. On a comparatively small sized diverse dataset consisting of 1147 images of 37 different signs, they achieved a validation set accuracy of 84.68%. To get good results with this small dataset, they used transfer learning and fed their data to a VGG16 network model pre-trained with ImageNet dataset. A large diverse dataset along with a deep learning-based recognition model was proposed in Rafi et al. (2019). They considered 38 BSL alphabets, used the pre-trained VGG19 transfer learning model for classification, and were able to achieve 89.60% test accuracy on their dataset.

Abedin, Prottoy, Moshruha, and Hakim (2021) proposed a novel approach to solve the problem of BSL recognition which they called as 'Concatenated BSL Network'. The specialty of the network is that it combines a CNN along with a pose estimation network to enhance overall performance. In their model, 'Openpose', a pretrained hand pose estimation algorithm, is used to extract positional features from the images. In Tasmere, Ahmed, and Hasan (2020), Tasmere et al. proposed a real time method that can classify BSL digits from video data. The training data was built by capturing images from real-time video frames. The moving objects from video data was detected using Gaussian Mixture Model (GMM). After that, the images were further pre-processed with blurring, thresholding, normalization before being fed to a deep CNN model. The system is 97.63% accurate in classifying sign digits.

Hasan and Ahsan (2019) proposed a BSL digits recognition method that uses Histogram of Oriented Gradient (HOG) feature based multi-class Support Vector Machine (SVM). HOG features were calculated from binary image data. These features were used to train a ONE VS ONE multi-class SVM for classification. The system performs with 94.74% accuracy while requiring less time for sign recognition. Basnin, Nahar, and Hossain (2021) presented a CNN-LSTM-based model for Bangla Sign Language recognition. A large dataset consisting of 13,400 images of 36 classes was used to train and validate the system. The images were thoroughly pre-processed using background subtraction, gray scale conversion, morphological erosion, and median filtering. The images were then fed to a customized CNN network where a time-distributed flattening layer separates the CNN model from the LSTM layer. The model achieved an accuracy of 88.5% with no overfitting.

Tabassum et al. (2020) proposed a HOG feature based single-handed Bengali Sign Language recognition system. They prepared a dataset of 1400 images and pre-processed them accordingly with histogram equalization, and lightness smoothing to improve the image quality. Hand portion of the images were segmented using YCbCr color space. Then, the images were converted to gray scale and HOG features were calculated. These features were used to train a K-Nearest Neighbor classifier. The model is 91.1% accurate in classifying Bengali Sign images.

Ahmed et al. (2019) proposed a complete computer vision-based system that can recognize Bangla Sign digits and translate them to Bangla speech. Deep CNN was chosen for the classification purpose. The maximum accuracy achieved by the system is 92%. Haque et al. (2019) proposed a model to classify the BSL alphabets. They used a self-built dataset consisting of 130 samples and achieved 77.88% recognition accuracy. The image data was pre-processed with size normalization, gray scale conversion, and histogram equalization. The principal component analysis (PCA) was used to extract features and reduce dimensionality in the data. A K-Nearest Neighbor classifier was trained using these features for classification.

Shanta, Anwar, and Kabir (2018) presented a Bangla Sign Language detection system using Scale Invariant Feature Transformation (SIFT)

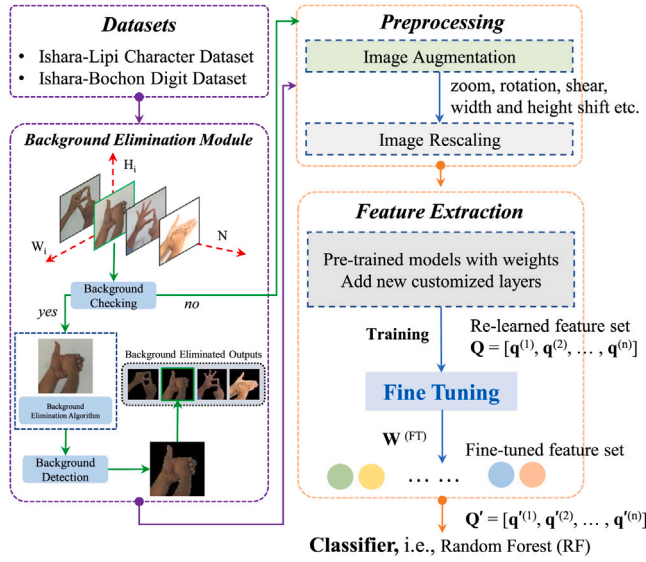


Fig. 1. Overall diagram of the proposed system, Q is the feature set to learn for the RF classifier, W is the weight matrix. W_i , H_i represent the width, and height of the input images respectively. N is the number of input samples.

and CNN. In the pre-processing step, the skin masking technique was used to segment the image to keep only the region of interest. An elliptical kernel was used to remove noise by performing erosion and dilations. Then, the edges of the images were detected using canny edge detection. Then, SIFT was applied to calculate scale (rotation, scaling, etc.) invariant features. These features were further clustered using k-means clustering before being fed to a CNN model for classification.

Khan, Joy, Asaduzzaman, and Hossain (2019) implemented a real-time, low-cost, fast Sign Language Translator prototype integrated into a Raspberry Pi device. It can detect 5 signs in real-time from video data coming through a webcam. At first, self build image dataset was created, augmented, and pre-processed accordingly. Then, the images were fed to a CNN classifier consisting of 3 convolutional, 3 normalizations, and 2 fully connected layers. After that, the model was integrated into a Raspberry Pi 3B. The accuracy while classifying signs from video data was approximately 94%.

2.4. Other sign languages

Apart from these, we have also studied Chinese Sign Language (Chai et al., 2013), German Sign Language (Dreuw, Deselaers, Keysers, & Ney, 2006), Korean Sign Language (Shin, Kim, & Jang, 2019), Japanese Sign Language (Sako & Kitamura, 2013), etc., to build a robust system that can recognize sign language accurately.

3. Methodology

Fig. 1 depicts the procedures involved in our sign language recognition system. The Ishara-Lipi datasets (Islam, Mousumi, Rabby et al., 2018; Islam, Mousumi, Rabby, Hossain et al., 2018) were used as the starting point. The images' backgrounds were eliminated in order to focus solely on the objects of interest. If the image dataset already consists of binary images then this step is skipped as the images are already well segmented. Then, necessary data augmentation and rescaling were done on the data. The data was then fed into the proposed model, which employed backbone networks as feature extractors and fed the extracted features to a random forest classifier. All of these steps resulted in a model that can successfully classify BSL characters and digit signs.

Algorithm 1: Background Elimination Algorithm

```

1 Input:  $In = \text{Input Images}$ 
2 Output:  $Out = \text{Background Eliminated Image}$ 
Data:
 $B = \text{Structuring Element (3} \times 3\text{)}$ 
 $RC = \text{Red Color Channel}$ 
 $BC = \text{Blue Color Channel}$ 
 $GC = \text{Green Color Channel}$ 
3 for each  $SubImage\ m \in InputImages$  do
4    $G_m \leftarrow \text{RGB\_To\_Gray}(SubImage)$ 
5    $F_m \leftarrow \text{Make\_Copy}(G_m)$ 
6   for  $i := 1 \rightarrow G_m.length$  do
7     for  $j := 1 \rightarrow G_m.width$  do
8        $V_{ij} \leftarrow \text{Adaptive\_Gaussian\_Threshold}(i, j)$ 
9       if  $G_m(i, j) > V_{ij}$  then
10         $F_m(i, j) \leftarrow 0$ 
11       else
12         $F_m(i, j) \leftarrow 255$ 
13    $O_m \leftarrow (F_m \ominus B_{(3 \times 3)}) \oplus B_{(3 \times 3)}$ 
14    $C_m \leftarrow (O_m \oplus B_{(3 \times 3)}) \ominus B_{(3 \times 3)}$ 
15    $RC_m, GC_m, BC_m \leftarrow \text{Split\_Channel}(SubImage)$ 
16   for  $i := 1 \rightarrow C_m.length$  do
17     for  $j := 1 \rightarrow C_m.width$  do
18       if  $C_m(i, j) := 0$  then
19          $BC_m(i, j) \leftarrow 0$ 
20          $GC_m(i, j) \leftarrow 0$ 
21          $RC_m(i, j) \leftarrow 0$ 
22    $Out_m \leftarrow \text{Merge\_Channel}(RC_m, GC_m, BC_m)$ 

```

3.1. Data preprocessing

3.1.1. Background elimination

Background elimination is applied to remove unnecessary features from the image data which does not contribute to the classification process. It detects the background from an image and replaces all the pixels of the background with black pixels, leaving only the object of interest in the image. The proposed approach for background elimination is presented in Algorithm 1 and the graphical illustration of the algorithm is shown in Fig. 2. The traditional technique for threshold calculation based on histogram, which occasionally creates some undesirable region, is also presented in this figure to demonstrate the superiority of the proposed algorithm. It is evident that the proposed algorithm's several stages are particularly effective at reducing various sorts of noise.

At first, we create a copy of the grayscale version of the original image. Secondly, we apply thresholding to the grayscale image. Thresholding is a technique in which the pixel value is compared with the threshold value, and the pixel value is then set based on the threshold value. The threshold value may be handpicked or calculated using some algorithm. In our case, we used the Gaussian adaptive threshold method. The thresholding operation outputs a binary image where black pixels and white pixels represent background and foreground respectively.

In basic thresholding, we had to manually preset a threshold value and in Otsu's thresholding, the optimal threshold value is automatically determined. But, both of these methods are global thresholding techniques, i.e. same threshold value is applied to segment all pixels in the image. Hence, it performs poorly in segmenting images that have variations in lighting and shading conditions in different local regions. Adaptive thresholding solves the problem by considering a small set of neighboring pixels at a time and computes the threshold

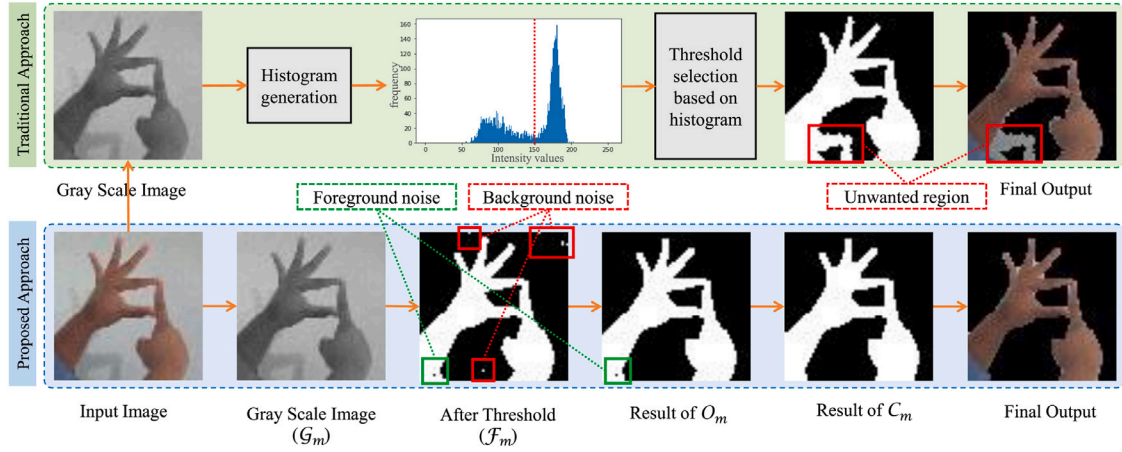


Fig. 2. Graphical illustration of algorithm 1. Symbols used in this figure are taken from algorithm 1. Undesirable regions created by histogram-based threshold calculation are also shown here.

based on that specific local region. So, the different threshold value is applied while thresholding different regions. In fact, the threshold value is calculated separately for each pixel of the image. Choosing the right size of pixel neighborhood is important as the thresholding largely depends on the assumption that, the neighboring pixels should have approximately uniform values. The neighborhood size should neither be too small to cover enough background and foreground pixels nor be too large to obey the assumption of local regions. The equation to calculate the threshold value for a pixel is defined in the Eq. (1).

$$T(i, j) = GMean(i, j) - C \quad (1)$$

Here, $T(i, j)$ is the threshold value calculated for the pixel at position (i, j) in the gray-scaled image. The mean value $GMean(i, j)$ calculated here is the Gaussian mean which is a weighted arithmetic mean where pixel values farther away from the centering pixel at (i, j) are assigned lesser weights. C is a constant value that tunes the threshold. There may be cases where the mean value alone is insufficient to distinguish between the background and foreground. The result of the threshold can be improved by adding or subtracting some constant value C . The value for C is entirely dependent on application and simple to tune by trial and error method. The expression for 2D Gaussian distribution is given in Eq. (2) where σ represents the standard deviation of the distribution and the distribution is assumed to have a mean of zero.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2)$$

Eq. (3) is used to transform the grayscale image into a binary image where $T(x, y)$ is a threshold that is computed for each pixel individually and I_{src} , I_{dst} denote the intensity value at (x, y) co-ordinates for the source and destination image respectively.

$$I_{dst}(x, y) = \begin{cases} 0 & \text{if } I_{src}(x, y) > T(x, y) \\ 255 & \text{otherwise} \end{cases} \quad (3)$$

After thresholding, we applied morphological opening and morphological closing transformations on the binary image to remove noise. A morphological opening of image A using the structuring element B is performed by executing two consecutive basic morphological transformations, an erosion operation followed by a dilation operation. The closing, on the other hand, is a dilation operation followed by an erosion operation. The equations of morphological erosion and dilation operations are stated in Eqs. (4) and (5) respectively.

$$A \ominus B = \{z | (B)_z \subseteq A\} \quad (4)$$

$$A \oplus B = \{z | [(B)_z \cap A] \subseteq A\} \quad (5)$$

A represents the set of foreground pixels and B is the structuring element. z 's are foreground values (1's represent foreground pixels and 0's represent background pixels). The equation of erosion inside the set of curly braces indicates that the erosion of A by B is the set of all points z , such that B , translated by z , is contained in A . This formulation results in the new foreground pixels of the image. Similarly, the dilation of A by B is the set of all displacements, z , such that the foreground elements of $\hat{B}$ overlap at least one element of A (Gonzalez & Woods, 2018).

Opening mostly helps to remove noise in the background and closing mostly helps to remove noise in the foreground. As a result, we get a binary image map. After that, we apply the 'logical AND' operation on each of the three color channels of the original image and the binary image map i.e. if the pixel value to the image map is 0, then we set that pixel value of each three-color channel to 0. Otherwise, we keep the pixel value as it is. Finally, we get a well-segmented color image where the background is removed by black pixels, and the foreground is preserved as it is.

3.1.2. Data augmentation

The datasets we are working with are relatively small. The character dataset comprises 1005 images and the digit dataset comprises 1075 images. But for deep learning, we need a larger dataset to get good results. For this purpose, we used the data augmentation technique to produce a larger dataset. In real-world applications, the input data can be very different from the training data. Usually, training data are created in an ideal condition. But the training data does not always reflect the same condition as real-world natural data. As a result, it is usually ideal to augment the dataset to a margin so that it can replicate real-world natural data as accurately as possible while also reducing the model's overfitting on the training set. The different types of augmentation techniques that are applied in our system include zoom, shear, horizontal flip, height and width shift, etc.

3.1.3. Data scaling

The stochastic gradient descent is used as the optimization technique in our model. Gradient descent is highly sensitive to the feature value. A high range of values of features causes irregular gradient descent updates. To ensure the update operation to be smooth, we normalize the feature value to a smaller range of 0 to 1 rather than a larger range of 0 to 255. For this, we multiplied all the pixels' values by $1/255$. It helps the algorithm to reach near the minima point in a lesser number of steps thus fastening the process.

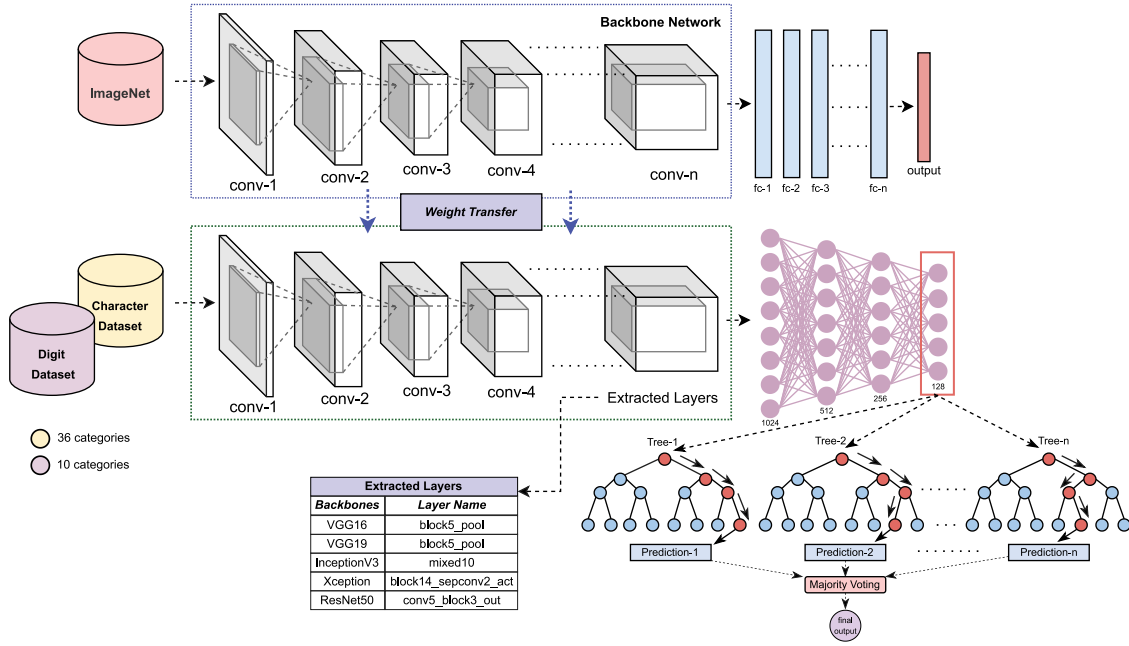


Fig. 3. Illustration of the proposed model architecture based on transfer learning.

Table 1
Summary of different extracted layers used in the proposed model.

Backbone networks	Extracted layers	Pre-trained	Character dataset		Digit dataset	
			Input shape	Parameters	Input shape	Parameters
VGG16	block5_pool	ImageNet	64 × 64 × 3	17,501,888	128 × 128 × 3	23,793,344
VGG19	block5_pool	ImageNet	64 × 64 × 3	22,811,584	128 × 128 × 3	29,103,040
InceptionV3	mixed10	ImageNet	64 × 64 × 3	24,589,984	128 × 128 × 3	30,881,440
Xception	block14_sepconv2_act	ImageNet	64 × 64 × 3	40,425,896	128 × 128 × 3	55,105,960
ResNet50	conv5_block3_out	ImageNet	64 × 64 × 3	32,666,368	128 × 128 × 3	57,832,192

3.2. Model description

A convolutional neural network-based approach is used for feature extraction in our system. CNN learns different features from images while training. Moreover, we applied the idea of transfer learning to assign the pre-trained weight in the model which helps enhance the overall performance.

3.2.1. Convolutional Neural Network

Convolutional Neural Network is a bio-inspired process that took motivation from the organization of the human visual cortex. The process revolves around a mathematical operation called convolution operation which operates on two functions, a portion of digital image and a feature detector called filter, resulting in another output function called feature map. The convolution operation for a 2D image I and a 2D filter F is represented in Eq. (6), where k is the kernel size. $[i, j]$ represents a specific position (i 'th row, j 'th column) in a 2D image. G is the resulting feature map.

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k I[u, v] F[i - u, j - v] \quad (6)$$

Different convolutional layers extract different types of features, such as initial layers of convolution are known to capture low-level features like edges of images, and later convolutional layers are known to capture high-level composite features. The process helps reduce the number of trainable parameters and successfully analyzes position and orientation dependency in an image. We used five different powerful CNN models pre-trained on 'ImageNet' as the backbone of our sign language recognition system.

3.2.2. Random forest

The random forest algorithm can be seen as an improvement of another supervised machine learning algorithm called the decision tree. The decision tree uses the concept of the n -ary tree to solve classification and regression problems. Each of the leaf nodes in the tree corresponds to a class label. Features are represented on the internal nodes that are called decision nodes that help generate predictions from a series of feature-based splits. Random forest uses a collection of decision trees to build the forest that overcomes the limitation of the decision tree algorithm which has the tendency to overfit training data. Random forest builds multiple random decision trees and merges them together to make the model more accurate and less sensitive to training data. A random forest model is trained with the 'Bagging method' which is a combination of two steps, bootstrapping and aggregation.

Let, the training set $X = \{x_1, x_2, \dots, x_n\}$ where x_i is a single training sample and corresponding label set, $Y = \{y_1, y_2, \dots, y_n\}$ where y_i is the label of x_i 'th training sample. Bootstrapping generates m new random training sets each of size n by random sampling with a replacement where repeated selections are allowed from the original training set X . This step ensures that we are using different datasets for every tree which makes the model less sensitive to training data. After that, m decision trees are built using each of these random datasets separately with a random subset of features considered from the set of all features. This step of choosing a random subset of features is called 'feature bootstrapping' which helps reduce the correlation between trees. These random decision trees are collectively labeled as the random forest. The prediction for a new data point is calculated using the aggregation technique where the data point is passed to each of the random decision trees. In the case of regression, after training, for a new test sample x , the resulted prediction is calculated by averaging the output predictions

produced by individual decision trees. On the other hand, in the case of classification, the resultant prediction is calculated by majority voting where the prediction of the maximum number of random decision trees is taken as the resultant prediction.

3.2.3. Backbone networks

VGG16 (Simonyan & Zisserman, 2014) is a CNN-based networks taking motivation from LeNet-5 architecture. VGG16 is a 16-layer neural network with 13 convolutional layers and 3 fully connected layers. Five max-pooling layers have a pooling window of size 2×2 pixel. All the convolutional filters are of size 3×3 and stride of 1. VGG19 (Simonyan & Zisserman, 2014) is quite similar to architecture of VGG16. The main difference is the number of convolutional filters is 13 for VGG16 and 16 for VGG19.

The specialty of the inception network (Szegedy, Vanhoucke, Ioffe, Shlens, & Wojna, 2016) is the inception modules that ensure efficiency by keeping the number of parameters small without hurting the performance. Instead of picking the size of the convolutional filter by hand, it uses various sized filters along with pooling in the same layer. Factorizing large-size filters into multiple smaller filters to reduce computation is also a noticeable contribution by the Inception network.

Xception network (Chollet, 2017) is a modification of inception network. In the case of the Xception network, the inception module from the inception network is replaced by depth-wise separable convolutions. Depth-wise separable convolutions are depth-wise convolutions followed by point-wise convolutions. A depth-wise convolution is applied to a single channel at a time rather than all the channels. Then a point-wise 1×1 convolution is applied to these channels. This change enhances the performance of the network on the ImageNet dataset. As both architectures have the same number of parameters, it can be concluded that the enhancement of performance is due to the more efficient use of model parameters.

Training very deep neural networks sometimes cause exploding or vanishing gradient problem that results in degradation of accuracy. That is where ResNet network (He, Zhang, Ren, & Sun, 2016) comes in handy with the idea of residual blocks or shortcut connections that take the advantages of residual learning. The main idea is to take the activation from one layer and feed it directly to a deep layer of the network. That allows very deep layers of a neural network to learn significant complex patterns successfully.

3.2.4. Transfer learning

The main idea of transfer learning (Pan & Yang, 2009) is to use the knowledge gained from one domain to perform another related task of a similar domain. It is often looked at as an optimization where machines can use previously learned knowledge or experience to solve a problem of similar feature-space distribution. Though it does not guarantee enhanced performance unless the knowledge comes from solving a generalized task. The modern world is producing an abundance of data but there is a scarcity of the labeled ones. Manually generating a large amount of labeled data is expensive and sometimes transfer learning is the only way when we do not have enough data to train a model.

Let, the domain be a two-element tuple, $D = \{\mathcal{X}, \mathcal{P}(\mathcal{X})\}$, where $\mathcal{X}$ is the feature space which is the set of all feature vectors of the training element. $\mathcal{P}(\mathcal{X})$ is a marginal probability distribution and $\mathcal{X}$ is a sample data point where $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$ and x_i is the i th feature vector corresponding to a sample data point. Similarly, let the task be a two-element tuple, $\mathcal{T} = \{\mathcal{Y}, f(x)\}$, where $\mathcal{Y}$ is the label space, set of all labels. $f(x), f : \mathcal{X} \rightarrow \mathcal{Y}$, is the objective predictive function that is not observed but learned from training using data points. The training data comprises of the collection of pairs $\{x_i, y_i\}$, where $x_i \in \mathcal{X}, y_i \in \mathcal{Y}$. The objective predictive function $f(x)$ is also denoted as a conditional probability distribution, $\mathcal{P}(y|x)$ from the probabilistic viewpoint. Given, a source domain DS and a target domain DT , a source learning task TS and a target learning task TT , where $DS \neq DT$, or $TS \neq TT$, the goal of transfer learning is to improve the learning of the target

objective predictive function $f_T(x)$ in DT using the knowledge from DS and TS .

There are generally two conventional approaches for using transfer learning. The first one is to define a custom CNN model architecture and train it with a large dataset of the same domain that performs a related task. Then, re-train the model with the dataset of the target task to get the final model. The other more popular approach that we used for our model is to select a pre-trained model created by researchers that perform greatly in benchmark datasets. The model is then fine-tuned accordingly to the target task and a portion of the model or the whole model is re-trained with the dataset of the target task to get the final model.

CNN is a great addition to the field of computer vision and deep learning. There are no standardized rules globally accepted for defining a specific kind of model. Scientists did years of experiments and research and came up with some powerful CNN models, such as VGG16, VGG19, ResNet50, InceptionV3, Xception, etc. These models when pre-trained on benchmark datasets such as ImageNet to initialize weights of trainable parameters become a generalized powerful image classifier. Then, these models are trained again using a specific dataset after necessary modification in architecture to enhance performance even more. Finally, the model becomes a specialized classifier of the dataset.

3.2.5. Model architecture

Different CNN architectures such as VGG16, VGG19, Xception, InceptionV3, and ResNet50 with 'ImageNet' weight are applied as the backbone networks which take the advantages of transfer learning and extract useful features from images. We extracted layer till *block5_pool*, *block5_pool*, *mixed10*, *block14_sepconv2_act*, and *conv5_block3_out* from VGG16, VGG19, InceptionV3, Xception, ResNet50 respectively for both character and digit recognition. Brief information about the backbone networks and extracted layers used for character and digit recognition is stated in Table 1. Some layers of the models were modified to fit our dataset. The input layer and the output layer (softmax) layer were changed, all the fully connected layers from the backbone network were removed and some custom fully connected layers were added according to our dataset. Lastly, a random forest classifier was added for the classification task. The architecture of the proposed model is presented in Fig. 3.

3.3. Model optimization and training

3.3.1. Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is one of the most common and popular optimization algorithms for optimizing neural networks. Although SGD means using an example at a time to calculate the gradient, it is also used interchangeably to refer to other kinds of gradient descents, mini-batch, and batch gradient descent. It is often used to minimize multi-variable objective function $F(x)$ which is differentiable. To be more precise, it is the 1st order repetitive optimization technique to find a local minima of a differentiable function. It does so by taking iterative steps towards the direction of the negative gradient of function $F(x)$. The main focus of the whole optimization process is to minimize the loss function. Eq. (7) states a single update of gradient descent. The gradient is denoted by the sign ' ∇ ', which is a measure of the rate of change of a variable with respect to another variable. After a single update from a point a_n , it gives a point a_{n+1} where $F(a_{n+1}) \leq F(a_n)$.

$$a_{n+1} = a_n - \gamma \nabla F(a_n) \quad (7)$$

Here, γ is called the learning rate that defines the size of each step which requires careful choice of value.

3.3.2. Categorical crossentropy

Categorical Cross-Entropy (CE) is an algorithm to calculate the loss of a multi-class classifier model, where the goal is to classify a single class from some number of classes. Here, the loss value represents the measure of the difference between two probability distributions. The higher the loss value, the larger the difference. Eq. (8) states the formula for calculating categorical cross-entropy loss. The two probability distributions are the distribution of the training dataset and the distribution of the model. The model is optimized accordingly using the loss value.

$$CE = - \sum_{i=1}^C t_i \log(s_i) \quad (8)$$

Here, C is the number of total classes, t_i is the ground truth and s_i is the CNN predicted score. It is also called log loss. It is preferred over other loss functions such as squared error loss because it gives a convex optimization problem. The significance of the minus sign is that the loss gets smaller when the ground-truth value and the CNN predicted score is closer to each other. The loss is calculated to compute the overall cost (J) of the model which is illustrated in Eq. (9). The loss function calculates the error for a single training example whereas the cost function evaluates performance error on the entire training set.

$$J = \frac{1}{m} \sum_{i=1}^m CE_i \quad (9)$$

The goal of calculating cost is to optimize the model to represent the same distribution of the training set. It is done by iteratively training the model by tuning the model parameters, so that the categorical cross-entropy loss may be minimized.

3.3.3. Model training and parameters

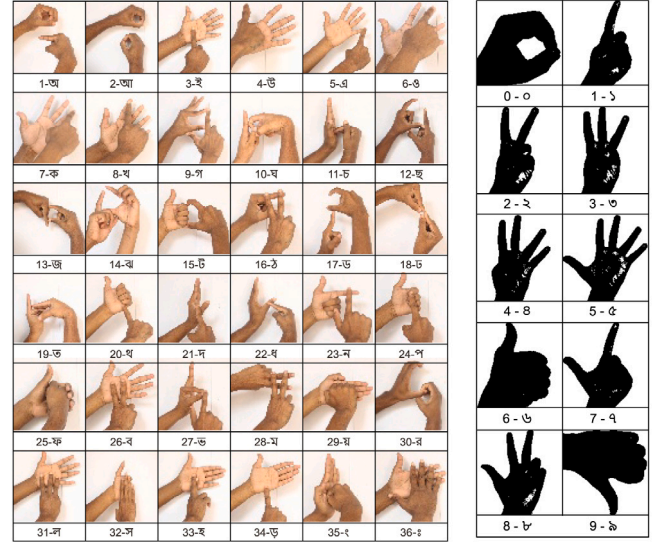
Consider, $\mathcal{N}$ training samples such as $\{(x_i, y_i)\}_{i=1}^{\mathcal{N}}$ where each sample $x_i \in \mathcal{R}^d$ is assigned a label $y_i \in \{l_1, l_2, \dots, l_{N_c}\}$. Every l_j represents a one-hot encoded vector such as $l_i = [l_{i,1}, \dots, l_{i,j}, \dots, l_{i,N_c}] \in \mathcal{R}^{N_c}$. In this vector, only the j th element is set to one and other elements contain zeros. Here, N_c denotes the number of classes or categories in the training set. The model is trained with mini batch technique where the initial weight is assigned from the pre-trained model previously trained on the ImageNet. Through training, the re-learned features set $Q = [q^{(1)}, q^{(2)}, \dots, q^{(n)}]$ is fine-tuned for feeding the RF classifier.

In the training phase, the model was trained with SGD optimizer while keeping the initial learning rate $1 \times e^{-2}$ and categorical cross-entropy was used to calculate the loss. The model was trained for 300 epochs as after that the loss curves become approximately flat. The optimizer, loss function and other model hyper-parameters were chosen through trial and error and parameter tuning.

4. Evaluation metrics

Generally, ‘accuracy’ is considered to be the single most popular metric to evaluate a model performance. However, it can be tricky sometimes and accuracy alone cannot establish a model’s resilience (Perrotta, 2020). So, we evaluated our model with accuracy along with other metrics such as precision, recall and F1-score. The formula for calculating these metrics are stated in Eqs. (10)–(13). To calculate these metrics, we require measures like true positive (TP), true negative (TN), false positive (FP) and false negative (FN). For multi-class classification, a TP event is one where a test example of class X is correctly predicted as class X by the classifier. FN is where a test example of class X is incorrectly predicted as some other class Y . A FP event refers to an event where some other class Y was incorrectly predicted as class X . Similarly TN refers to an event where a test example of some other class Y was correctly predicted to be not a test example of class X .

$$Accuracy = \frac{(TP + TN)}{(TP + FN + FP + TN)} \quad (10)$$



(a) Ishara-Lipi character dataset.

(b) Ishara Bochon digit dataset.

Fig. 4. Bengali sign language datasets.

$$Precision = \frac{TP}{(TP + FP)} \quad (11)$$

$$Recall = \frac{TP}{(TP + FN)} \quad (12)$$

$$F1 - Score = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)} \quad (13)$$

5. Experiments

NumPy, Pandas, Scikit-learn, TensorFlow (version: 2.0), and Python 3 (version: 3.6) have been utilized to develop the proposed system. The system is implemented on a core-i7 processor with NVIDIA GeForce GTX 1050 Ti GPU (CUDA Cores: 768) and 16 GB of physical memory. The developed models are also tested using the ‘Google Colab’ platform, which makes use of the NVIDIA Tesla K80 GPU.

5.1. Dataset collection and description

The models were tested on two different datasets. Accessing both datasets from Islam, Mousumi, Jessan and Hossain (2018), we obtained 1075 and 1005 image samples for Bangla sign digit and Bangla sign characters respectively. For digit dataset (Ishara-Bochon), each image has a resolution of 128×128 pixels and is saved as a .jpg file. The character dataset (Ishara-Lipi) originally had over 1800 samples, however, it currently contains only 1005 samples. Bengali language contains 50 basic characters (11 vowels and 39 consonants) (Reza & Khan, 2012). However, some of the characters are not used as much as others. The character dataset contains 36 Bengali sign characters (6 vowels and 30 consonants) featuring most of the important phonemic characters where each of the 36 character classes includes on average 28 sign images. The images are taken from various angles, lighting conditions, and hand positions. These images are colored (RGB) images of size 64×64 pixels each and saved in .jpg format. Examples of each class for both datasets are presented in Fig. 4 with appropriate labels.

5.2. Result analysis and discussion

Five different extended and modified pre-trained models were used to train both digit and character sign datasets. The training was performed separately on two different datasets.

Table 2

Detailed results for different combinations of backbone networks and different batch sizes.

Methods	Batch size	Character dataset					Digit dataset				
		Accuracy (in %)	Precision (in %)	Recall (in %)	F1-score (in %)	Training time (in s)	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-score (in %)	Training time (in s)
VGG16	10	88.33	91.38	88.33	88.16	996.35	96.67	97.50	96.67	96.75	3044.03
	20	83.89	86.62	83.89	83.84	894.94	97.33	97.67	97.33	97.31	2856.00
	30	87.22	89.89	87.22	86.73	827.12	96.00	96.64	96.00	96.01	2741.11
	40	87.78	90.36	87.78	87.43	815.46	97.33	97.57	97.33	97.30	2767.79
	50	85.56	89.85	85.56	84.95	1002.92	96.67	97.27	96.67	96.68	3269.83
	60	86.67	89.79	86.67	86.23	783.34	96.00	96.46	96.00	96.00	2659.02
	70	85.00	86.93	85.00	84.44	799.80	96.00	96.46	96.00	95.95	2692.61
VGG19	10	86.67	87.86	86.67	85.90	1054.45	96.67	97.27	96.67	96.68	3320.35
	20	80.00	84.76	80.00	79.72	933.66	96.67	97.27	96.67	96.70	2985.95
	30	84.44	86.43	84.44	83.36	912.02	94.67	95.01	94.67	94.71	2892.77
	40	82.22	85.36	82.22	81.81	1023.66	96.00	96.88	96.00	96.03	3336.25
	50	91.11	92.94	91.11	90.97	830.92	96.67	97.50	96.67	96.75	2769.02
	60	81.67	82.58	81.67	80.60	798.99	96.67	97.11	96.67	96.71	2726.07
	70	78.33	81.78	78.33	76.05	799.58	96.67	97.08	96.67	96.62	2712.30
InceptionV3	10	77.78	83.16	77.78	77.50	1566.90	96.67	97.18	96.67	96.68	3256.14
	20	78.33	79.85	78.33	76.98	1187.67	90.00	90.86	90.00	89.91	2919.20
	30	75.56	76.35	75.56	73.74	1087.15	91.33	91.79	91.33	91.20	2763.61
	40	74.44	78.87	74.44	73.94	1045.60	97.33	97.57	97.33	97.27	2684.85
	50	73.33	77.37	73.33	72.55	1004.26	95.33	95.79	95.33	95.37	2680.07
	60	72.78	75.98	72.78	71.80	970.58	91.33	92.54	91.33	91.42	2631.59
	70	77.22	79.87	77.22	77.14	965.22	96.67	97.08	96.67	96.64	2598.27
Xception	10	76.67	80.22	76.67	75.96	1422.40	95.33	95.93	95.33	95.34	3427.56
	20	77.22	81.22	77.22	75.86	1199.27	92.00	92.95	92.00	92.00	3378.89
	30	75.56	81.39	75.56	75.29	1079.87	92.67	94.09	92.67	92.74	2980.19
	40	72.22	76.46	72.22	71.52	1018.43	94.67	95.39	94.67	94.70	3027.95
	50	76.11	79.23	76.11	75.54	1145.72	87.33	89.12	87.33	87.02	3066.49
	60	74.44	78.35	74.44	74.49	982.41	90.67	91.88	90.67	90.56	2854.70
	70	72.78	76.69	72.78	72.44	967.77	92.67	93.45	92.67	92.68	2912.54
ResNet50	10	81.67	81.95	81.67	80.22	1315.74	94.67	95.06	94.67	94.72	3306.01
	20	77.22	82.04	77.22	76.63	976.00	96.00	96.88	96.00	96.08	2922.11
	30	70.56	71.90	70.56	68.84	1085.61	97.33	97.71	97.33	97.31	3140.90
	40	70.56	78.00	70.56	71.15	862.71	94.00	95.16	94.00	93.99	2747.33
	50	66.67	71.09	66.67	65.60	987.65	92.00	92.77	92.00	92.05	2935.59
	60	66.67	69.91	66.67	65.30	812.71	88.67	90.51	88.67	88.74	2693.82
	70	71.67	73.09	71.67	70.34	794.07	94.00	94.61	94.00	94.01	2724.05

To verify the effectiveness of the RF classifier on the features extracted by different pre-trained models, we trained different models without and with the RF classifier, as shown in Tables 2 and 3 respectively. In most cases, models with the RF classifier perform better than models without the RF classifier. The best performing model from Table 2 is VGG19 (batch size: 50) for character recognition and ResNet50 (batch size: 30) for digit recognition. The VGG19 (batch size: 50) model for character recognition was able to achieve 91.11%, 92.94%, 91.11%, 90.97% of accuracy, precision, recall, and f1-score respectively. The ResNet50 (batch size: 30) model for digit recognition, on the other hand, attained the accuracy, precision, recall, and f1-score of 97.33%, 97.71%, 97.33%, and 97.31% respectively. If we take a closer look at Table 3, it is evident that the model VGG16 with RF classifier surpasses other models in performance for both character and digit recognition.

For character recognition, the hybrid model (VGG16 + RF) with batch size 40 achieved the best results with accuracy, precision, recall, and f1-scores of 91.67%, 93.64%, 91.67%, 91.47%, and for digit recognition, the model with batch size 20 delivers the highest results with accuracy, precision, recall, and f1-scores of 97.33%, 97.89%, 97.33%, 97.37%, respectively. Besides, this model takes a reasonable training time of 820.46 s and 2858.73 s for character and digit recognition respectively.

Though both the character and the digit datasets are from the same authors, they are two separate datasets with different features and characteristics. Thus, their distribution is also different. As shown in Table 2, VGG19 performs best for character recognition. On the other hand, ResNet50 performs best for digit recognition. VGG16 and VGG19 were able to obtain good results in character recognition, with accuracies of over 80% for nearly all batch sizes and models based

on other backbone networks failed to produce good results, with most accuracies falling below 80%. For digit recognition, all the models based on different backbone networks with various batch sizes perform satisfactorily. The worst performing models are ResNet50 (batch size: 60) without RF and Xception (batch size: 50) without RF classifier for character and digit recognition respectively as they achieve the least performance metrics i.e. accuracy, precision, recall, and f1-scores of (66.67%, 69.91%, 66.67%, 65.30%) and (87.33%, 89.12%, 87.33%, 87.02%) for character and digit recognition respectively.

Gradient-weighted Class Activation Mapping or Grad-CAM is a heatmap visualization process for a given class categories. Table 4 indicates the Grad-CAM visualization by highlighting the important regions of the images. It is the localization map that allows us to see where the model is looking in order to categorize the images. The heatmaps of the Grad-CAM are superimposed on the original images for better understanding.

Fig. 5 illustrates the 'accuracy vs. epoch' and 'loss vs. epoch' graphs for different backbone networks. From the graphs, it is evident that all the models based on different backbone networks are free of overfitting as the accuracy is increasing and loss is decreasing with the increase of each epoch. Besides, the ROC curve (Receiver Operating Characteristic curve) and precision-recall curve for the proposed model are shown in Fig. 6. For most of the categories, the area covered by both the ROC curve and precision-recall curve is 1.00 or very close to 1.00, which signifies the high ability of our model to distinguish between different classes. It proves that our model is robust enough to give a good evaluation for test samples. t-Distributed Stochastic Neighbor Embedding (t-SNE) is a non linear technique for visualizing high dimensional data. It represents each high-dimensional entity with a two or three-dimensional point in such a way that comparable things are described

Table 3

Detailed results from the hybrid model for different combinations of backbone networks and batch sizes. The hybrid model is developed by feeding the features of different layers from various backbone networks to the Random Forest (RF) classifier.

Methods	Batch size	Character dataset					Digit dataset				
		Accuracy (in %)	Precision (in %)	Recall (in %)	F1-score (in %)	Training time (in s)	Accuracy (in %)	Precision (in %)	Recall (in %)	F1-score (in %)	Training time (in s)
Hybrid (VGG16 + RF)	10	88.89	91.59	88.89	88.87	1000.73	96.00	96.88	96.00	96.03	3046.53
	20	88.33	89.50	88.33	88.17	899.42	97.33	97.89	97.33	97.37	2858.73
	30	90.00	91.87	90.00	89.95	831.73	96.67	97.27	96.67	96.68	2743.58
	40	91.67	93.64	91.67	91.47	820.46	96.67	97.08	96.67	96.62	2770.23
	50	88.33	91.46	88.33	87.66	1008.24	95.33	96.02	95.33	95.34	3272.87
	60	88.89	91.05	88.89	88.44	788.39	95.33	96.02	95.33	95.34	2661.51
	70	85.00	87.80	85.00	83.84	804.51	97.33	97.71	97.33	97.31	2695.20
Hybrid (VGG19 + RF)	10	88.89	90.31	88.89	88.39	1058.47	96.67	97.50	96.67	96.75	3322.82
	20	83.33	86.59	83.33	82.88	937.75	96.00	96.88	96.00	96.08	2988.48
	30	89.44	91.28	89.44	89.27	916.93	96.67	97.13	96.67	96.71	2895.98
	40	87.78	89.70	87.78	87.54	1028.82	96.00	96.88	96.00	96.03	3339.29
	50	91.67	93.62	91.67	91.25	835.43	96.67	97.50	96.67	96.75	2771.87
	60	85.56	87.49	85.56	85.29	803.69	95.33	96.03	95.33	95.42	2728.95
	70	85.00	86.58	85.00	84.38	803.97	96.67	97.50	96.67	96.75	2715.12
Hybrid (InceptionV3 + RF)	10	83.89	87.58	83.89	83.84	1570.68	96.00	96.73	96.00	96.03	3258.51
	20	82.78	85.14	82.78	81.94	1192.22	94.67	95.42	94.67	94.68	2922.08
	30	77.22	79.59	77.22	76.53	1092.19	94.67	94.98	94.67	94.65	2765.99
	40	78.33	80.06	78.33	78.17	1050.69	96.67	97.08	96.67	96.62	2687.40
	50	77.22	80.42	77.22	76.12	1009.43	94.00	95.23	94.00	94.11	2683.21
	60	74.44	76.71	74.44	73.35	975.65	94.67	95.39	94.67	94.70	2634.08
	70	75.00	77.48	75.00	74.66	970.47	96.00	96.64	96.00	96.01	2601.03
Hybrid (Xception + RF)	10	80.00	83.13	80.00	79.95	1426.48	96.67	96.99	96.67	96.62	3430.20
	20	78.89	84.51	78.89	78.29	1204.12	95.33	96.52	95.33	95.44	3381.67
	30	78.89	81.59	78.89	78.72	1084.75	96.67	96.94	96.67	96.63	2983.09
	40	78.89	82.85	78.89	78.96	1023.53	94.67	95.48	94.67	94.70	3030.53
	50	77.78	81.48	77.78	76.77	1151.07	92.67	93.31	92.67	92.67	3069.98
	60	75.56	79.27	75.56	74.90	988.29	94.67	95.32	94.67	94.68	2857.86
	70	73.33	77.99	73.33	73.05	973.57	97.33	97.71	97.33	97.34	2915.26
Hybrid (ResNet50 + RF)	10	85.00	86.66	85.00	84.81	1320.09	96.67	96.88	96.67	96.63	3308.16
	20	82.78	85.57	82.78	83.04	980.06	96.00	96.44	96.00	96.07	2924.38
	30	73.33	75.48	73.33	73.08	1090.47	96.67	97.08	96.67	96.62	3143.73
	40	75.00	79.31	75.00	75.12	867.60	95.33	95.88	95.33	95.35	2749.93
	50	72.78	75.41	72.78	71.71	992.19	97.33	97.71	97.33	97.34	2938.39
	60	76.11	77.69	76.11	75.14	817.36	95.33	96.02	95.33	95.34	2696.28
	70	76.67	79.07	76.67	76.42	799.13	95.33	95.83	95.33	95.37	2726.40

Table 4

Grad-CAM visualization of different images showing important area of image for classification where S_n represents different test samples.

Image Label	Character Dataset				Digit Dataset			
	S_1	S_2	S_3	S_4	S_1	S_2	S_3	S_4
Background Eliminated Images								
Heatmap								
Grad-CAM								

by adjacent points with a high probability, while different objects are modeled by distant points. For giving a reasonable demonstration of the datasets, we plotted a t-SNE visualization in Fig. 7.

The detailed comparison between the existing systems and the proposed BSL recognition system is depicted in Table 5. Many of the systems have chosen neural network as the classifier. It is also evident from Table 5 that accuracy tends to fall when the number of classes increases. Systems working with only digit recognition were able to achieve higher accuracy than the systems working with character recognition. According to Table 5, VGG19 (batch size: 50) offers an accuracy of 97.33% for numerals and ResNet50 (batch size: 30) provides an accuracy of 91.11% for characters. On the other hand,

the accuracy of the proposed hybrid model (VGG16 + RF) is 97.33% for numerals and 91.67% for characters. First of all, our goal is to develop a generalized model that can produce higher accuracy for both datasets. Secondly, we also have to consider the number of parameters that the model needs to learn during the training phase to produce the result. From Table 1, it is evident that the total trainable parameters for VGG16, VGG19 and ResNet50 are 17,501,888, 22,811,584, 32,666,368 for character dataset and 23,793,344, 29,103,040, 57,832,192 for digit dataset respectively. For both datasets, VGG16 has fewer trainable parameters than VGG19 and ResNet50. As shown in Tables 2 and 3, it is also apparent that the proposed model takes less training time. To be specific, the training time for the proposed model is 820.46 s for

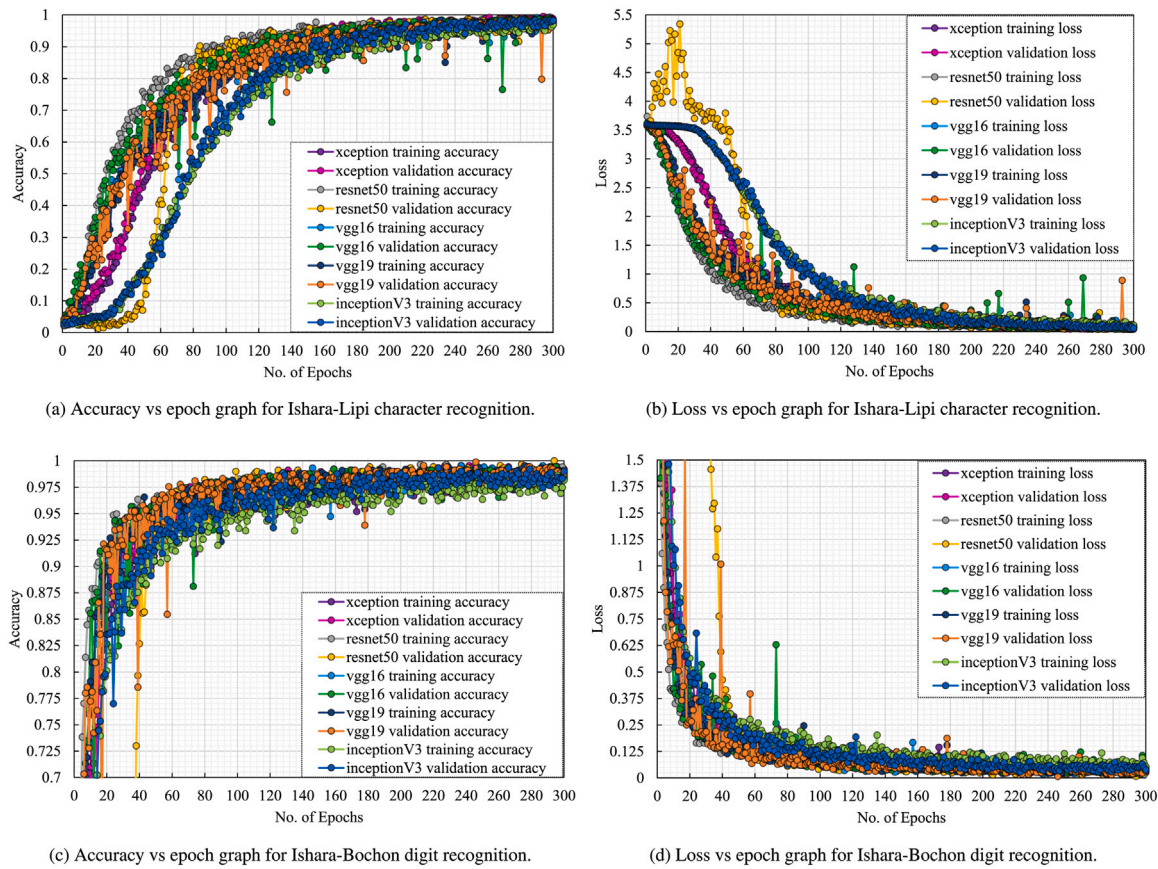


Fig. 5. Visualization of accuracy and loss graph with respect to no. of epochs for both character and digit recognition.

numerals and 2858.73 s for character dataset, whereas, the training time for individual VGG19 (batch size: 50) and ResNet50 (batch size: 30) is 830.92 s for numerals and 3140.90 s for character dataset which are significantly longer than the training time of the proposed model. As the proposed model produces satisfactory accuracy for both datasets with less trainable parameters and less training time, the proposed hybrid model (VGG16 + RF) is better than the individual VGG19 or ResNet50 model.

The proposed system outperforms most of the existing systems in performance. Due to the unavailability of the complete Ishara-Lipi character dataset (Islam, Mousumi, Jessan, Rabby and Hossain, 2018; Islam, Mousumi, Rabby et al., 2018), we used currently existing 1005 samples. If we compare other systems that used the same dataset as ours, the proposed system surpasses Hasan and Ahsan (2019) and Islam, Mousumi, Rabby, Hossain et al. (2018) in digit recognition by achieving an accuracy of 97.33% whereas Hasan and Ahsan (2019) and Islam, Mousumi, Rabby, Hossain et al. (2018) achieved an accuracy of 95% and 94.74% respectively. In the case of character recognition, the proposed system achieved 91.67% accuracy and Islam, Mousumi, Rabby et al. (2018) achieved 92% accuracy. Although Islam, Mousumi, Rabby et al. (2018) obtained better accuracy than the proposed system, we must keep in mind that the proposed system utilized only $(1005/1800) \times 100 = 55.83\%$ of the original Ishara-Lipi character dataset and reached $(91.67/92) \times 100 = 99.64\%$ of the system's accuracy proposed by Islam, Mousumi, Rabby et al. (2018), as shown in Table 6 and Fig. 8. Similarly, we analyzed other systems that exceed the proposed system and summarized the results in Table 6. From Table 6, we can undoubtedly say that the proposed system employed a limited amount of dataset and achieved most of the other system's accuracy.

The major goal of this research is to use an effective background elimination strategy and present a hybrid model that combines transfer learning and random forest to get better results. By automating the

process of categorizing sign language, this model might serve as a bridge between the normal and hearing-impaired communities. If the hearing impaired and normal people can work together in the same place with no communication gap, a huge socio-economic change can happen.

6. Conclusion

This paper presented a hybrid transfer learning-based deep learning system with the RF classifier for Bangla sign language recognition. In the pre-processing phase, the proposed background elimination algorithm removes unnecessary background information from images, allowing the model to focus on the region of interest and thus improve the model's inference speed. The proposed system provides a reliable architecture for smaller datasets as it uses pre-trained models, at the same time, the fine-tuning process with the lower learning rate enhances the model's performance while preventing overfitting. Furthermore, the RF classifier utilizes random subsets of the fine-tuned features and averages several decision trees to reduce the variance of the final model caused by a single decision tree.

All of these aspects contributed to the development of the system that outperformed many of the state-of-the-art recognition systems in terms of accuracy. However, we feel that the results of character recognition could be improved. The data pre-processing such as background elimination produces better outcomes than data with no pre-processing. In the future, we plan to build a smartphone-based system for the hearing impaired community. For that, a reduced model architecture with a small set of parameters will be required to cope with the smartphone's limited computational resources and power. We also have a plan to create a high-quality, complete, and open-source BSL dataset for future researchers.

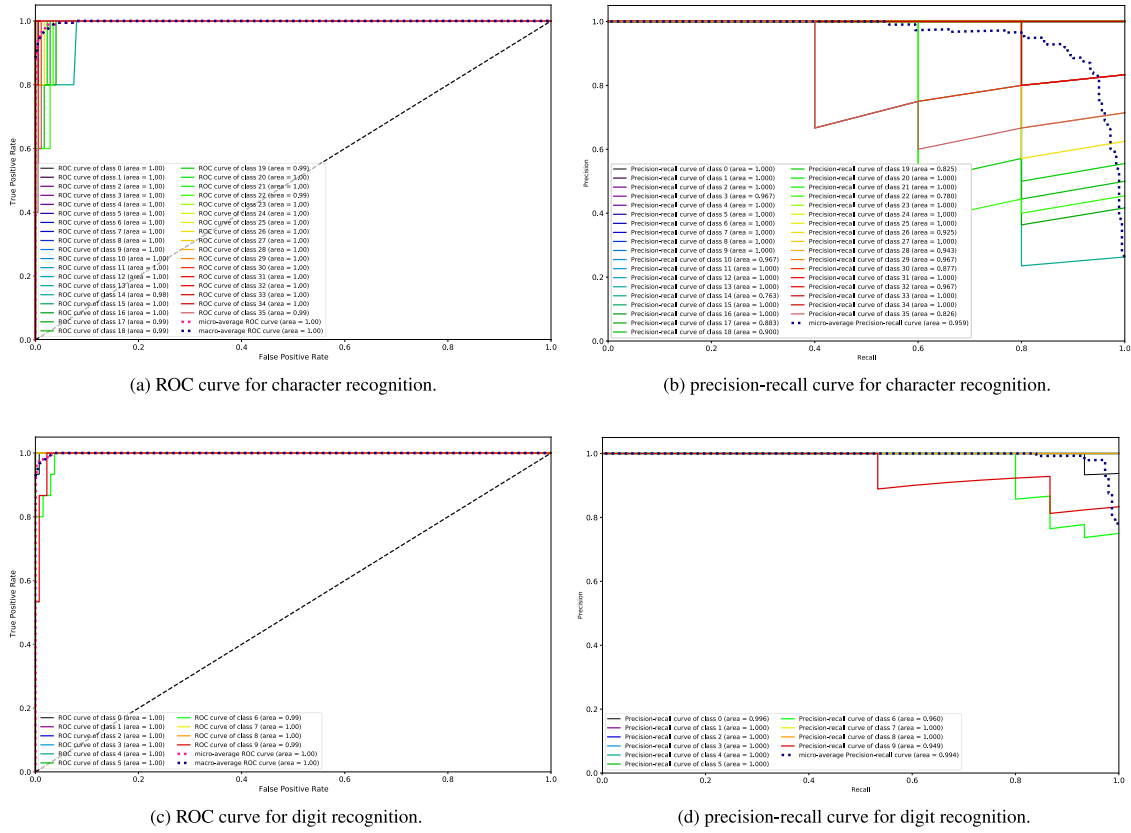


Fig. 6. Multi-class ROC AUC and multi-class precision-recall AUC on Ishara-Lipi character and Ishara-Bochon digit dataset respectively where, the area under the curve (AUC) is a scalar statistic to evaluate the model. The ROC plot is generated by projecting the true-positive rate on the y-axis and the false-positive rate on the x-axis. On the contrary, the precision-recall plot is produced by projecting the precision on the y-axis and the recall on the x-axis.

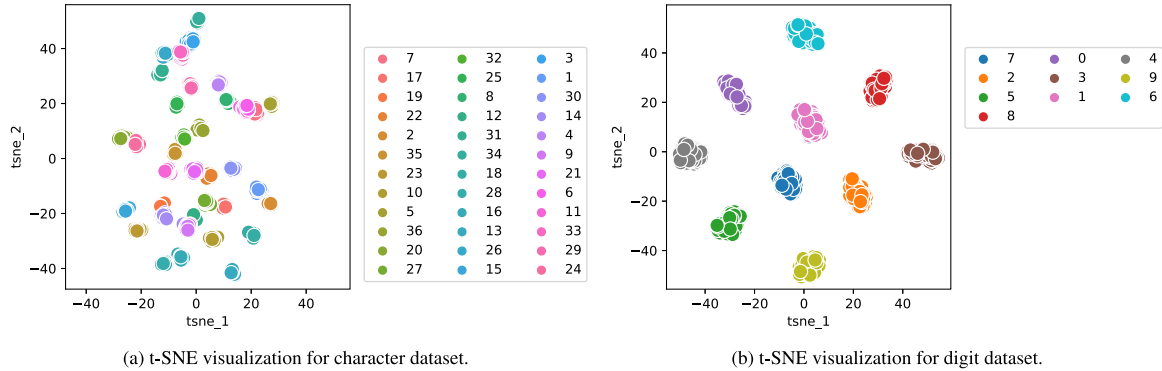


Fig. 7. Color coded t-SNE visualization for both character and digit dataset.

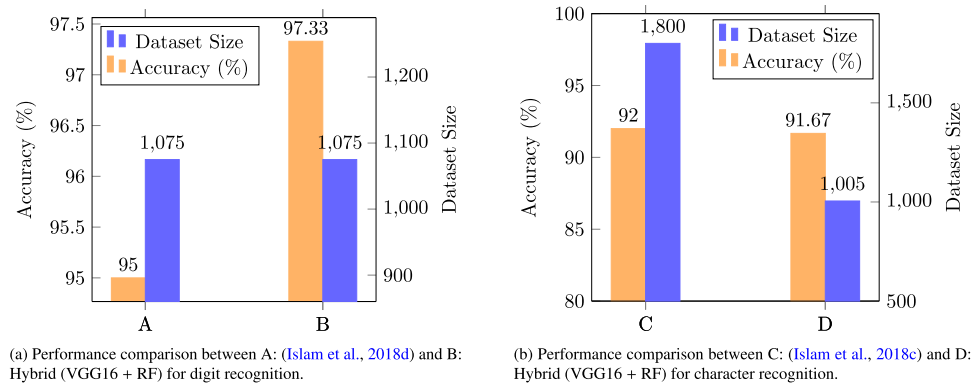


Fig. 8. Visual comparison between the proposed system and other systems that used same datasets.

Table 5

Comparison among existing and the proposed BSL system for Bengali sign language recognition. The average accuracy is expressed as a percentage for ease of understanding.

Developed BSL systems	Methodology	Dataset	Class	Average accuracy
Islam, Mousumi, Rabby, Hossain et al. (2018)	CNN	Ishara-Bochon: 1075 samples	10 numerals	95%
Islam, Mousumi, Rabby et al. (2018)	CNN	Ishara-Lipi: 1800 samples	36 characters	92%
Rahman et al. (2012)	ANN and image processing	828 samples (train: 540, test: 288)	36 alphabets	80.90%
Uddin et al. (2017)	YCbCr color model, Bag of features and SVM	800 samples	15 characters	86%
Rony et al. (2018)	CNN, Inception-v3	2050 samples (alphabets: 1900 samples, special characters: 150 samples)	38 alphabets, 3 special characters	92.85%
Jarman et al. (2015)	Fingertip finder algorithm with multilayered feed forward neural network	2300 samples	46 hand gestures (9 numerals, 9 vowels, 28 consonants)	88.69%
Hossen et al. (2018)	Deep CNN	1147 samples	37 characters	84.68%
Rafi et al. (2019)	VGG19	12,581 samples	38 alphabets	89.60%
Abedin et al. (2021)	Concatenated BdSL network (CNN based image network + pose estimation network)	12,581 samples (Train: 9959, validation: 1102, test: 1520)	38 alphabets	91.51%
Tasmere et al. (2020)	CNN	1674 samples (Train: 1066, validation: 271, test: 337)	10 numerals	97.63%
Hasan and Ahsan (2019)	HOG feature and multi-class Support Vector Machine	Ishara-Bochon: 1000 samples (train: 900, test: 100)	10 numerals	94.74%
Basnin et al. (2021)	Integrated CNN-LSTM	13,400 samples	36 alphabets	88.5%
Tabassum et al. (2020)	HOG features and KNN	1400 samples	35 alphabets	91.1%
Ahmed et al. (2019)	CNN	3200 samples	10 numerals	92%
Haque et al. (2019)	Principal Component Analysis (PCA) and KNN algorithm	130 samples (train: 104, test: 26)	26 alphabets	77.88%
Shanta et al. (2018)	Scale Invariant Feature Transform (SIFT) and CNN	1700 samples	38 characters	90.63%
Khan et al. (2019)	CNN and customized ROI Segmentation	500 training samples, after augmentation: 4000 samples	5 sign gestures	94%
Ours: Best result from Table 2	VGG19 (batch size: 50) ResNet50 (batch size: 30)	Ishara-Bochon: 1075 samples Ishara-Lipi: 1005 samples	10 numerals 36 characters	97.33% 91.11%
Ours (proposed): Best result from Table 3	Hybrid (VGG16 + RF) Hybrid (VGG16 + RF)	Ishara-Bochon: 1075 samples Ishara-Lipi: 1005 samples	10 numerals 36 characters	97.33% 91.67%

Table 6

Relative comparison between the proposed BSL system with other systems that ostensibly exceeds our system in performance with respect to percentage of dataset and percentage of accuracy.

BSL systems	Recognition	% of dataset	% of accuracy
Islam, Mousumi, Rabby et al. (2018)	Character	55.83	99.64
Tasmere et al. (2020)	Digit	64.22	99.69
Rony et al. (2018)	Character	49.02	98.73

CRediT authorship contribution statement

Sunanda Das: Conceptualization, Methodology, Investigation, Software, Validation, Writing – original draft, Writing – review & editing.
Md. Samir Imtiaz: Resources, Software, Data curation, Visualization.
Nieb Hasan Neom: Resources, Software, Writing – original draft, Editing.
Nazmul Siddique: Writing – review & editing, Supervision.
Hui Wang: Writing – review, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The dataset used in this research is an open-source dataset.

References

- Abedin, T., Prottoy, K. S., Moshruha, A., & Hakim, S. B. (2021). Bangla sign language recognition using concatenated BdSL network. *arXiv preprint arXiv:2107.11818*.
- Ahmed, S., Islam, M., Hassan, J., Ahmed, M. U., Ferdosi, B. J., Saha, S., et al. (2019). Hand sign to bangla speech: A deep learning in vision based system for recognizing hand sign digits and generating bangla speech. *arXiv preprint arXiv:1901.05613*.
- Anand, K., Urolagin, S., & Mishra, R. K. (2021). How does hand gestures in videos impact social media engagement-Insights based on deep learning. *International Journal of Information Management Data Insights*, 1(2), Article 100036.
- Aziz, K. E., Wadud, A., Sultana, S., Hussain, M. A., & Bhuiyan, A. (2017). Bengali sign language recognition using dynamic skin calibration and geometric hashing. In *2017 6th international conference on informatics, electronics and vision & 2017 7th international symposium in computational medical and health technology (ICIEV-ISCMT)* (pp. 1–5). IEEE.
- Barczak, A., Reyes, N., Abastillas, M., Piccio, A., & Susnjak, T. (2011). *A new 2D static hand gesture colour image dataset for ASL gestures*. Massey University.
- Basnin, N., Nahar, L., & Hossain, M. S. (2021). An integrated CNN-LSTM model for bangla lexical sign language recognition. In *Proceedings of international conference on trends in computational and cognitive engineering* (pp. 695–707). Springer.
- Berlitz (2021). The most spoken languages in the world.
- Bilgin, M., & Mutludoğan, K. (2019). American sign language character recognition with capsule networks. In *2019 3rd international symposium on multidisciplinary studies and innovative technologies ISMSIT*, (pp. 1–6). IEEE.
- Chai, X., Li, G., Lin, Y., Xu, Z., Tang, Y., Chen, X., et al. (2013). Sign language recognition and translation with kinect. In *IEEE conf. on AFGR*, Vol. 655 (p. 4).
- Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1251–1258).
- Das, S., Fime, A. A., Siddique, N., & Hashem, M. (2021). Estimation of road boundary for intelligent vehicles based on DeepLabV3+ architecture. *IEEE Access*, 9, 121060–121075.
- Dreuw, P., Deselaers, T., Keyers, D., & Ney, H. (2006). Modeling image variability in appearance-based gesture recognition. In *ECCV workshop on statistical methods in multi-image and video processing* (pp. 7–18).

- Fok, K.-Y., Cheng, C.-T., & Ganganath, N. (2015). Live demonstration: A hmm-based real-time sign language recognition system with multiple depth sensors. In *2015 IEEE international symposium on circuits and systems ISCAS*, (p. 1904). IEEE.
- Gonzalez, R., & Woods, R. (2018). *Digital image processing*. Pearson.
- Haque, P., Das, B., & Kaspy, N. N. (2019). Two-handed bangla sign language recognition using principal component analysis (PCA) and KNN algorithm. In *2019 international conference on electrical, computer and communication engineering ECCE*, (pp. 1–4). IEEE.
- Hasan, M. M., & Ahsan, S. M. M. (2019). Bangla sign digits recognition using HOG feature based multi-class support vector machine. In *2019 4th international conference on electrical information and communication technology EICT*, (pp. 1–5). IEEE.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
- Hossen, M., Govindaiah, A., Sultana, S., & Bhuiyan, A. (2018). Bengali sign language recognition using deep convolutional neural network. In *2018 Joint 7th international conference on informatics, electronics & vision (iciev) and 2nd international conference on imaging, vision & pattern recognition (icIVPR)* (pp. 369–373). IEEE.
- Iandola, F. N., Han, S., Moskewicz, M. W., Ashraf, K., Dally, W. J., & Keutzer, K. (2016). SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size. arXiv preprint arXiv:1602.07360.
- Islam, M. S., Mousumi, S. S., Jessan, N. A., & Hossain, S. A. (2018). The ISHARA-LIPI dataset of bangla sign language digits and characters. <https://isharalipi.sanzidscloud.com/>.
- Islam, M. S., Mousumi, S. S. S., Jessan, N. A., Rabby, A. S. A., & Hossain, S. A. (2018). Ishara-lipi: The first complete multipurpose open access dataset of isolated characters for bangla sign language. In *2018 international conference on bangla speech and language processing ICBSLP*, (pp. 1–4). IEEE.
- Islam, M. S., Mousumi, S. S. S., Rabby, A. S. A., & Hossain, S. A. (2018). A simple and mighty arrowhead detection technique of bangla sign language characters with CNN. In *International conference on recent trends in image processing and pattern recognition* (pp. 429–437). Springer.
- Islam, M. S., Mousumi, S. S. S., Rabby, A. S. A., Hossain, S. A., & Abujar, S. (2018). A potent model to recognize bangla sign language digits using convolutional neural network. *Procedia Computer Science*, 143, 611–618.
- Jarman, A. M., Arshad, S., Alam, N., & Islam, M. J. (2015). An automated bangla sign language recognition system based on fingertip finder algorithm. *International Journal of Electronics & Informatics*, 4(1), 1–10.
- Kasukurthi, N., Rokad, B., Bidani, S., Dennisan, D., et al. (2019). American sign language alphabet recognition using deep learning. arXiv preprint arXiv:1905.05487.
- Khan, S. A., Joy, A. D., Asaduzzaman, S., & Hossain, M. (2019). An efficient sign language translator device using convolutional neural network and customized ROI segmentation. In *2019 2nd international conference on communication engineering and technology ICCET*, (pp. 152–156). IEEE.
- Kieffer, B., Babaie, M., Kalra, S., & Tizhoosh, H. R. (2017). Convolutional neural networks for histopathology image classification: Training vs. using pre-trained networks. In *2017 seventh international conference on image processing theory, tools and applications IPTA*, (pp. 1–6). IEEE.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Machiraju, S., Urolagin, S., Mishra, R. K., & Sharma, V. (2021). Face mask detection using keras, opencv and tensorflow by implementing Mobilenetv2. In *2021 3rd international conference on advances in computing, communication control and networking (ICAC3N)* (pp. 1485–1489). IEEE.
- Masood, S., Thuwal, H. C., & Srivastava, A. (2018). American sign language character recognition using convolution neural network. In *Smart computing and informatics* (pp. 403–412). Springer.
- Pan, S. J., & Yang, Q. (2009). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359.
- Perrotta, P. (2020). The problem with accuracy. <https://levelup.gitconnected.com/the-problem-with-accuracy-3670891b908e>.
- Rafi, A. M., Nawal, N., Bayev, N. S. N., Nima, L., Shahnaz, C., & Fattah, S. A. (2019). Image-based bangla sign language alphabet recognition for deaf and dumb community. In *2019 IEEE global humanitarian technology conference GHTC*, (pp. 1–7). IEEE.
- Rahman, M. A., Abdullah, M., Mondal, S., et al. (2012). Recognition of static hand gestures of alphabet in bangla sign language. *IOSR Journal of Computer Engineering (IOSRJCE)*, 8(1), 07–13.
- Rao, G. A., Syamala, K., Kishore, P., & Sastry, A. (2018). Deep convolutional neural networks for sign language recognition. In *2018 conference on signal processing and communication engineering systems SPACES*, (pp. 194–197). IEEE.
- Reza, K. N., & Khan, M. (2012). Grouping of handwritten Bangla basic characters, numerals and vowel modifiers for multilayer classification. In *2012 international conference on frontiers in handwriting recognition* (pp. 325–330). IEEE.
- Rony, A. J., Saikat, K. H., Tanzeem, M., & Robi, F. R. H. (2018). An effective approach to communicate with the deaf and mute people by recognizing characters of one-hand bangla sign language using convolutional neural-network. In *2018 4th international conference on electrical engineering and information & communication technology (ICEEICT)* (pp. 74–79). IEEE.
- Sajanraj, T., & Beena, M. (2018). Indian sign language numeral recognition using region of interest convolutional neural network. In *2018 second international conference on inventive communication and computational technologies ICICCT*, (pp. 636–640). IEEE.
- Sako, S., & Kitamura, T. (2013). Subunit modeling for japanese sign language recognition based on phonetically depend multi-stream hidden markov models. In *International conference on universal access in human-computer interaction* (pp. 548–555). Springer.
- Shanta, S. S., Anwar, S. T., & Kabir, M. R. (2018). Bangla sign language detection using sift and cnn. In *2018 9th international conference on computing, communication and networking technologies ICCCNT*, (pp. 1–6). IEEE.
- Sharma, C. M., Tomar, K., Mishra, R. K., & Chariar, V. M. (2021). Indian sign language recognition using fine-tuned deep transfer learning model. In *Procc. of international conference on innovations in computer and information science ICICIS*, (pp. 62–67).
- Shin, H., Kim, W. J., & Jang, K.-a. (2019). Korean sign language recognition based on image and convolution neural network. In *Proceedings of the 2nd international conference on image and graphics processing* (pp. 52–55).
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition.
- Singh, P., Mishra, R. K., Urolagin, S., & Sharma, V. (2021). Enhancing security by identifying facial check-in using deep convolutional neural network. In *2021 3rd international conference on advances in computing, communication control and networking (ICAC3N)* (pp. 1006–1010). IEEE.
- Sruthi, C., & Lijiya, A. (2019). Signet: A deep learning based indian sign language recognition system. In *2019 international conference on communication and signal processing ICCSP*, (pp. 0596–0600). IEEE.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2818–2826).
- Tabassum, T., Mahmud, I., Uddin, M. P., Emran, A., Afjal, M. I., & Nitu, A. M. (2020). Enhancement of single-handed bangla sign language recognition based on hog features. *Journal of Theoretical and Applied Information Technology*, 98(5), 743–756.
- Tasmere, D., Ahmed, B., & Hasan, M. M. (2020). Bangla sign digits: A dataset for real time hand gesture recognition. In *2020 11th international conference on electrical and computer engineering ICECE*, (pp. 186–189). IEEE.
- Uddin, J., Arko, F. N., Tabassum, N., Trisha, T. R., & Ahmed, F. (2017). Bangla sign language interpretation using bag of features and support vector machine. In *2017 3rd international conference on electrical information and communication technology EICT*, (pp. 1–4). IEEE.
- W. H. O. (2021). Deafness and hearing loss. <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>.
- Zaki, M. M., & Shaheen, S. I. (2011). Sign language recognition using a combination of new vision based features. *Pattern Recognition Letters*, 32(4), 572–577.